

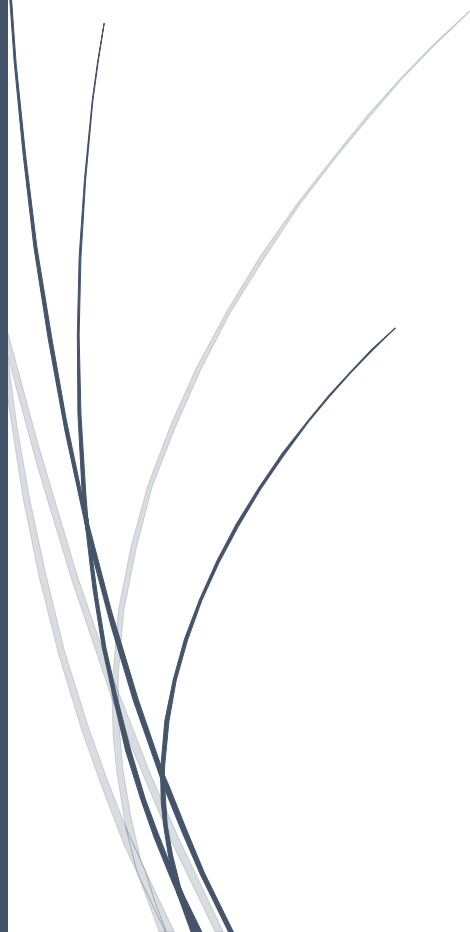


Rapport Projet C

BOUVIER D'ACHER VICTOR 26-A
JOUVANCEAU DAMIEN 26-B
MAYANS TOM 26-B

Graphes et compression de données

Un voyageur de commerce



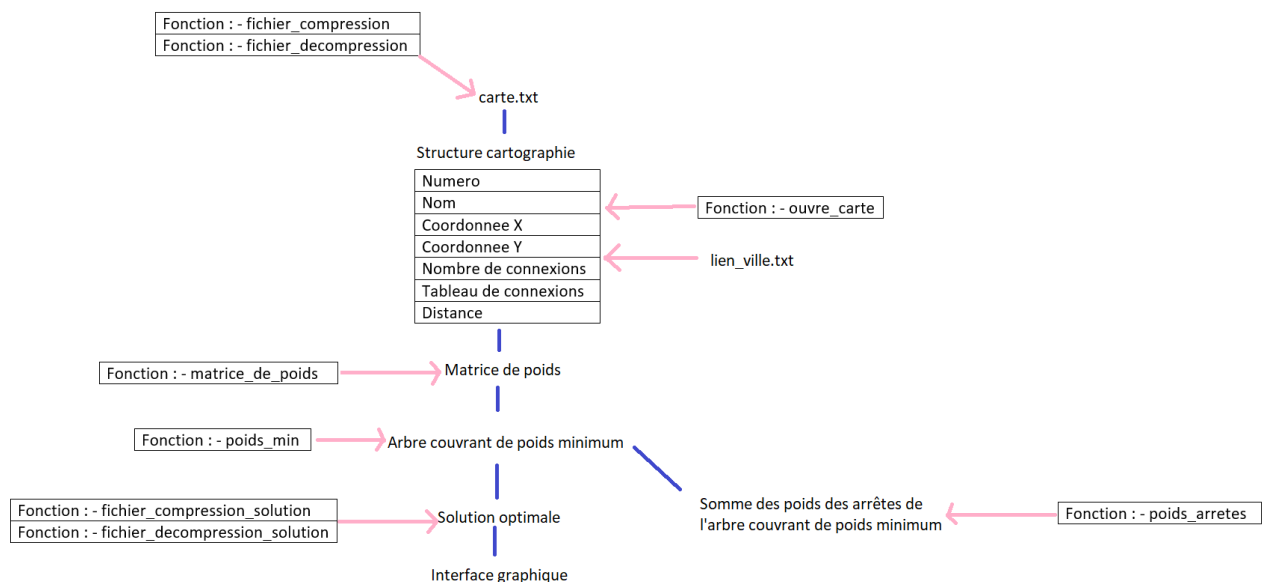
Introduction

Le but de ce projet est de trouver l'itinéraire du voyageur qui lui coutera le moins d'essence. Nous disposons d'un fichier `carte.txt` qui nous servira de base dans le projet. Dans le projet, nous répondrons au problème posé en utilisant les graphes ainsi que différents algorithmes. Nous ferons aussi de la compression et décompression de données.

Afin de comprendre le problème et gagner du temps sur la partie programmation, nous ferons une modélisation du problème.

Enfin, en dernier lieu nous utiliserons une interface graphique afin de visualiser les différentes étapes du projet.

Modélisation



Notre modélisation représente les différentes étapes que nous devons mener dans le projet afin de trouver la solution au problème posé.

Ainsi on observe ici que le programme partira du fichier `carte.txt`. A l'aide de ce fichier, on remplira une structure « Cartographie » dans laquelle on stockera toutes les données utiles liées aux villes. La structure nous permettra ainsi d'obtenir la matrice de poids et de continuer le programme, étape par étape, jusqu'à obtenir la solution optimale. L'ensemble est affiché sur l'interface graphique afin de suivre le bon déroulement du programme.

Gestion du projet

Tout d'abord, comme pour le projet précédent, nous avons divisé le travail en 3 parties. La première partie consistait à réaliser la modélisation et à visualiser correctement les différentes étapes du projet afin de, également, diviser la partie programmation entre les 3 membres du groupe. La deuxième partie concernait la programmation. Nous avons divisé les fonctions à réaliser en plus ou moins 3 parties égales afin d'avancer le projet à une vitesse convenable. La dernière partie servirait à résoudre les éventuels problèmes rencontrés lors de la programmation et à rendre le programme plus optimal.

Pour réaliser la modélisation, nous nous sommes principalement aidés du sujet du projet afin de faire la première version. En effet, nous modifions la modélisation au fur et à mesure que nous avançons sur la programmation afin qu'elle soit assez précise, notamment sur le nom des fonctions utilisées. Le projet étant assez linéaire et explicite, cette partie du projet n'a relativement pas posé de problèmes.

La deuxième partie du projet comprenait toute la partie programmation. C'est par conséquent celle qui nous a posé le plus de problèmes, ce qui fait sens. Nous nous sommes répartis le travail de sorte que tout le monde ait une partie sur laquelle travailler. Victor s'est occupé des questions, Damien des questions 1, 2 et 10, Tom des questions 3, 4 et 5. Bien sûr, ce n'était qu'une ligne directrice et nous étions libre d'aider l'un ou l'autre.

La première difficulté fut tout d'abord de lire correctement la carte. Pour vérifier les 1 dans le fichier. En effet, le 1 dans le fichier pouvait être suivi par un espace ou un autre numéro. Notre solution a été de créer une variable qui comprend 3 états : 0, 1 et 2. Si l'état était à 0, cela veut dire qu'il n'y avait pas de 1 à la dernière entrée, si l'état était à 1, alors il y avait un 1 à la dernière entrée. Enfin si l'état était à 2, c'était pour forcer le programme à aller à la fin de la boucle pour passer à l'état 0 et recommencer le processus.

Une autre difficulté était de choisir correctement l'algorithme pour calculer l'arbre couvrant de poids minimum. Nous avons choisi l'algorithme de Sollin mais il s'est révélé complexe car il nous fallait créer un graphe. Nous avons donc choisi l'algorithme de Kruskal puisqu'il demandait uniquement de créer plusieurs étapes de tri.

Pour la compression de carte.txt nous avons choisi Huffman. Nous avons donc suivi les étapes présentes dans le cours. Le problème était de stocker les différentes valeurs pour la compression puisque la récursivité n'est pas le plus pratique pour remplir des tableaux. Nous avons donc utilisé des fichiers externes pour capturer les valeurs au fur et à mesure et les ont réunis dans un seul fichier à la fin.

La décompression de `carte.txt` ne fut pas plus facile car nous n'arrivions pas à la recréer. Notre solution fut de recalculer la valeur en binaire de chaque chiffre et les mettre dans une matrice. La matrice nous servirait ensuite à réécrire la carte dans un autre fichier.

La troisième partie fut plus longue qu'habituellement puisque la répartition sur 3 personnes de la programmation a rendu le programme fastidieux à exécuter. Nous avons donc décidé à ce moment-là de faire plusieurs fichiers `.c` et non pas seulement 2 fichiers `main.c` et `fonctions.c`. Cela rendra le `makefile` plus lourd mais serait moins compliqué à faire marcher. De plus, cela rendrait le programme plus lisible en limitant le nombre de lignes pour chaque fichier.